Introduction to HTML and CSS?

→ HTML (Hyper text Markup Language) is the standard markup language used to create and structure content on webpages. It defines the basic structure of a webpages using elements and tags such as headings, paragraphs, links, images, lists, tables, forms and other multimedia content. HTML documents are interpreted by web browers to display the content in a structured and meaningful manner. HTML provides the structure and semantic organization of a webpage.

→ CSS (Cascading Style Sheets) is a stylesheet language used to control the presentation and visual appearence of HTML documents. It is used to define properties such as colors, fonts, backgrounds, spacing, borders, alignment, dimensions, and layouts. CSS also supports responsive design allowing webpages to adapt to different screen sizes and devices. CSS can be applied using inline, internal, & external stylesheets.

→ HTML and CSS work together to develope webpages, where HTML provides the structure and content, while CSS provides the styling, layout, visual presentation. The combination of both forms the foundation of modern web development and enables the creation of organized attractive and user-friendly web interfaces.

Difference between MERN Stack and FULL Stack

→ MERN Stack is a javascript based technology stack used for developing modern web applications. MERN stands for mongoDB, Express.js, React.js, Node.js, mongoDB is used for database management, Express.js handles the backend framework and API's, React.js provides the server side runtime environment.

→ FULL-Stack Development refers to the development of both the frontend and backend of a web application along with databases integration. A full stack developer works with technologies used for designing the user interface, implementing server-side logic, managing databases.

HTML Components:

1. HTML Tags - Define the structure and elements of a webpage
2. Headings - used to display titles and headings using <h1> to <h6>
3. paragraphs - used to display text using the <p> tag
4. Links - used to navigate between webpages using <a> tag
5. Images - used to display images using <image> tag
6. Lists - used to create ordered and unordered lists using <ol>, <ul>,
7. Tables - used to organize data in rows, and columns.
8. Forms - used to collect user input using elements such as text boxes, buttons and check boxes.
9. Div and Span - used to group and organize webpage content.
10. Semantic Elements - Elements such as <header> <nav>, <section>, <article>, <footer>

CSS Components

1. Selectors - Select HTML elements to apply styl
2. properties - Define the visual characteristics of Elements.
3. Values - Specify the setting of a CSS prop
4. Colors - Define text, background, border colors.
5. fonts - Control font family, size, style and weight
6. Backgrounds - Set background colors and images
7. Borders - Define the border style, width and radius.
8. Margin - Controls the space outside an Elem
9. Padding - Controls the space inside Element
10. Layout - Controls the arrangment of elements using Flexbox, grid and positioning

HTML VS HTML 5

HTML	HTML5
Older version of HTML	Latest major version of HTML
Limited multimedia support	Supports audio and video
Uses External plugins for multimedia	No external plugs needed
Has fewer semantic tags	provides Semantic tags
No <Canvas> Element	Supports <Canvas>
Limited web storage	Supports local storage
Complex DOCTYPE declaration	Simple <!DOCTYPE HTML>

CSS VS CSS3

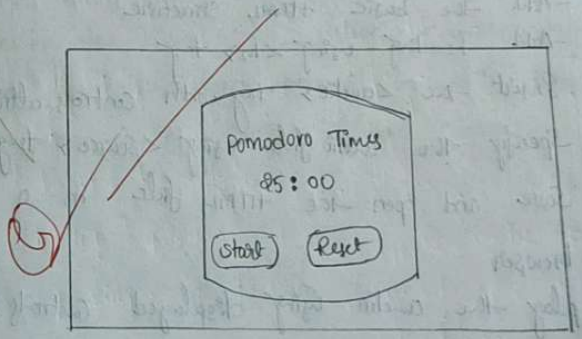
CSS	CSS3
Earlier version of CSS	Advanced version of CSS
Basic styling features	provides advanced styling features
Limited animations	Supports animations and transitions.
No flexible layout systems	Supports flexbox and Grid
Limited visual effects	Supports Shadows, gradients, effects
No media queries	Supports media queries
Uses a single specification	Divided into different modules.

Design a static webpage using HTML and CSS

Pomodoro Timer using HTML, CSS

```
<!DOCTYPE html>
<html>
<head>
    <title> pomodoro Timer </title>
    <style>
        body {
            font-family : Arial;
            back-ground : #f5f5f5;
            text-align : center
            padding : 50px;
        }
        .timer {
            width : 300px;
            margin : auto;
            padding : 30px;
            background : white;
            border-radius : 15px;
            box-shadow : 0 0 10px gray;
        }
        h1 {
            color : #e74c3c;
        }
        .time {
            font-size : 50px;
            margin : 20px;
        }
        button {
            padding : 10px 20px;
            margin : 5px;
            border : none;
            border-radius : 5px;
            background : #e74c3c;
            color : white;
        }
    </style>
</head>
<body>
    <div class = "Timer">
        <h1> Pomodoro Timer </h1>
        <div class = "time"> 25:00 </div>
        <button> start </button>
        <button> Reset </button>
    </div>
</body>
</html>
```

o/p:-



i) Design an HTML Code using audio tags

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>-Audio Example</title>
</head>
<body>
    <h2> My Favorite Song</h2>
    <audio controls>
        <source src="Song.mp3" type="audio/mpeg">
        your browser does not Support the audio tag
    </audio>
</body>
</html>
```

O/p:- Displays an audio player with play, pause and Volume Controls.

Steps:

Create a new HTML file

-Add the basic HTML Structure

-Add heading using <h2> tag

Insert the <audio> tag with controls attribute

-Specify the audio file using <source> tag

Save and open the HTML file in a web browser

play the audio using displayed Controls.

ii) Design a HTML Code using video tags

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title> Video Example </title>
</head>
<body>
    <h2> my video </h2>
    <video width="400" controls>
        <source src="video.mp4" type="video.mp4">
    </video>
</body>
</html>
```

O/p:- Display a video player with play, pause, Volume, and Full Screen Controls.

steps:-

Create a new HTML file

-Add a basic HTML Structure

-Add a heading using the <h2> tag

Insert the <video> tag with controls attribute

Specify the video file using <source> tag

Save the file and open it in web browser

play the video using displayed controls.

Write a program using Javascript (arrays, functions)

Code:

```
let numbers = [12,45,8,23,56,19,34,67,90,11]
function displayArray(arr){
        Console.log("Array Elements:");
        for(let i=0; i<arr.length; i++){
            Console.log(arr[i]);
        }
}
function findSum(arr){
        let sum=0;
        for(let i=0; i<arr.length; i++){
            Sum+=arr[i];
        }
        return Sum;
}
```

EXPERIMENT - 04

04 Develop dynamic webpage using Java Script

program

```
<!DOCTYPE html>
<html>
<head>
    <title> Dynamic webpage </title>
    <style>
        body {
            text-align: center;
            font-family: Arial;
            margin-top: 100px;
        }
        button{
            padding: 10px 200px;
            font-size: 16px;
            cursor: pointer
        }
    </style>
</head>
<body>
    <h1 id="heading"> Welcome to my webpage
    </h1>
    <p id="message"> click the button to change
        this webpage </p>
    <button onclick="changePage()"> click me
    </button>
<script>
```

```
function changepage() {
document.getElementById("heading").
        innerHTML = "welcome to javascript!";
document.getElementById("message").
        innerHTML = "The webpage has been changed
                    dynamically.";
document.body.style.backgroundColor = "light blue";
}
</script>
</body>
</html>
```

Output

Welcome to my webpage
click the button to change this webpage

click me

When the user clicks "click me", the javascript function changepage() is called, then the heading dynamically changes and background also changed

After clicking the button

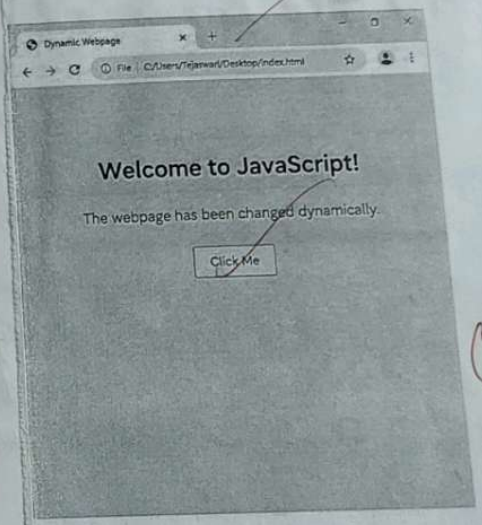
Welcome to Javascript!

The webpage has been changed dynamically

click me



Before clicking the button



After clicking the button

EXPERIMENT - 05

05. Implement Modules in Node.JS

In Node.js you can implement modules using two distinct systems : CommonJS (JS), which is traditional default. and ESmodules (ESM). the modern javaScript standard.

1. The CommonJS method (default)

CommonJS uses modules.exports to share code and require() to import it this System loads modules Synchronously

a. Create the module (math.js) :

```
Const add = (a,b) => a+b;
Const Subtract = (a,b) => a-b;
module.exports = {
    add,
    Subtract
};
```

b. use the module (app.js)

```
Const math = require('./math');
Consale.log(math.add(10,5));
Congale.log(math.Subtract(10,5));
```

Output : 15

5

2. The ES modules Method (modern standard):

ES modules use export to share code and import to use it to make Node.js recognize ES modules, you must either use the .mjs file extension or add "type": "module" to your package.json file.

a. Create the module (calculator.js):

```
Export const multiply = (a,b) => a*b
Export const divide = (a,b) => a/b
const calculator Name = "Super Calc";
Export default calculator Name;
```

b. Use the module (main.js)

```
import calculator Name, {multiply, divide}
                    from './calculator.js';
Console.log (Calculator Name);
Console.log (multiply (4,2));
Console.log (divide (4,2));
```

Output:- Super Calc
8
2

3. Using Built-in core modules

Node.js also comes equipped with dozens of built-in core modules (like for file systems or path for file routing) that don't need installation we will load them by module names:

```
// CommonJS style
const fs = require ('fs');
// ES modules style
import fs from 'fs';
```

Here are some of the most commonly used one's;

1. fs = file System operations.
2. https - HTTPS server and client
3. os - operating System utilities
4. path - file path utilities
5. events - Event handling
6. util - utility functions
7. Stream - Stream handling
8. Crypto - Cryptographic functions
9. Url - URL parsing
10. querystring - URL query string handling

Example:- Simple HTTPS server

```
http.Create server (req, res) => {
    res.writeHead (200, {'Content-Type':
                         'text/html'});
    res.End ('Hello World!');
}).listen (8080);
```

Output:-
Hello World!

EXPERIMENT - 06

06. Develope mini application using express and NodeJs
. Simple mini application using node JS + ExpressJS

Student management System

1. Add a Student
2. View all Students
3. delete a Student
4. uses fxpress.js routes
5. Simple HTML interfaces

project Structure

```
Student-app/
├── server.js
└── public
    └── index.html
```

Server.js

```
Const express = require ("express");
Const app = Express ();
Const PORT = 3000;
let students = [];
app.use (express json ());
app.use (express. Static ("public"));
app.get ("/Student", (req, res) => {
        res.json (Students);
    });

//Add
app.post ("/Students", (req, res) => {
        const Student = {
        id: Students. length + 1,
    name : req. body. name;
```

```
course : req.body.course
};
students.push(student);
res.json({message: "Student added successfully",
          student: student});
});
// Delete
app.delete("/students/:id", (req, res) => {
    const id = parseInt(req.params.id);
    students = students.filter(student =>
      student.id !== id);
    res.json({
      message: "Student deleted successfully"});
});
// Start server
app.listen(PORT, () => {
    console.log('server running at https://localhost:
    ${PORT}');
});
```

public/index.html:

```
<!DOCTYPE html>
<html>
<head>
    <title> Student management </title>
</head>
<body>
    <h1> Student management System </h1>
    <input type="text" id="name"
    placeholder="Enter Student name">
    <input type="text" id="course"
    placeholder="enter course">
    <button onclick="addStudent()"> Add Student
    </button>
    <h2> Student list </h2>
    <ul id="studentlist"></ul>
    <script>
      function addstudent(){
        let name = document.getElementById("name").
                   value;
        let course = document.getElementById("course").
                     value;
        fetch("/students", {
          method: "post",
          headers: {
            "Content-Type": "application/json"},
          body: JSON.stringify({
                name: name,
                course: course
            })
          })
          .then(response => response.json())
          .then(data => {
                alert(data.message);
document.getElementById("name").value = "";
document.getElementById("course").value = "";
          displaystudents();
        });
    }
```

```
function displayStudents() {
    fetch("/students").then(response => response.j
    .then(students => {
    let list = document.getElementById("studentList
    list.innerHTML = "";
    students.forEach(student => { let li =
        document.createElement("li");
        li.innerHTML = student.id + " - " +
                       student.name + " - " +
                       student.course + " <button
    onclick='deleteStudent(" + student.id + ")'>Delete
    </button>";
    list.appendChild(li);
            });
        });
    }
    function deleteStudent(id) {
        fetch("/students/" + id, { method: "DELETE" })
    .then(response => response.json())
    .then(data => {
            alert(data.message);
            displayStudents();
        });
    }
    displayStudents();
</script>
</body>
</html>
```

Output :-

Student Management System

Enter Student name

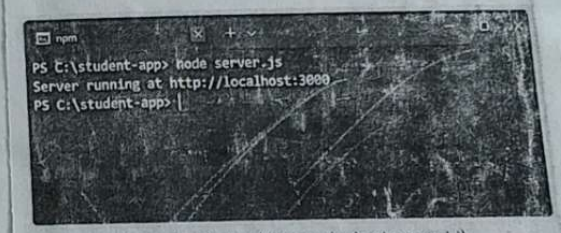
Enter Course

Add Student

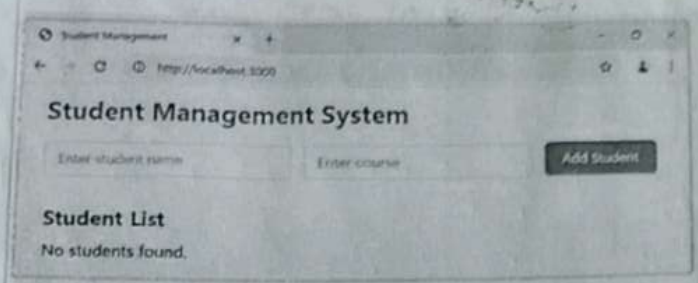
Student list :-

1. Ram - CSM [delete]
2. Sita - CSE [delete]

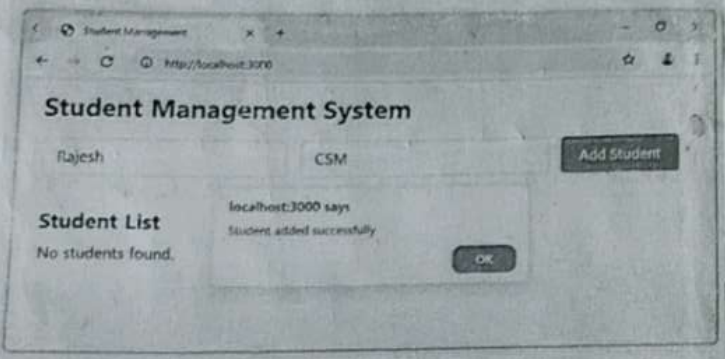
- The Express.js server is successfully started at http://localhost:3000
- The browser displays the student management system with fields.



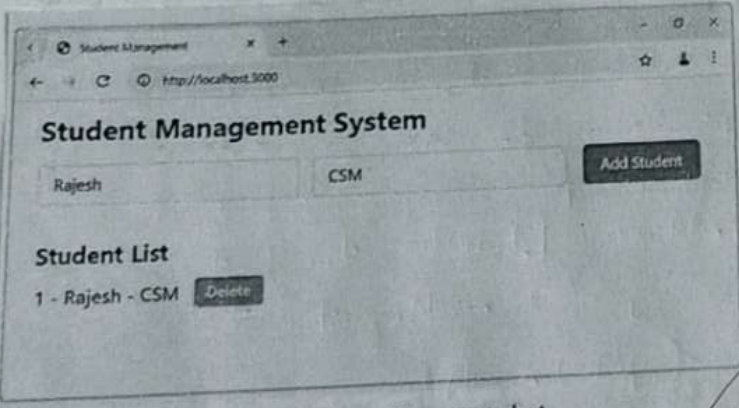
1. Terminal Output (after running `node server.js`)



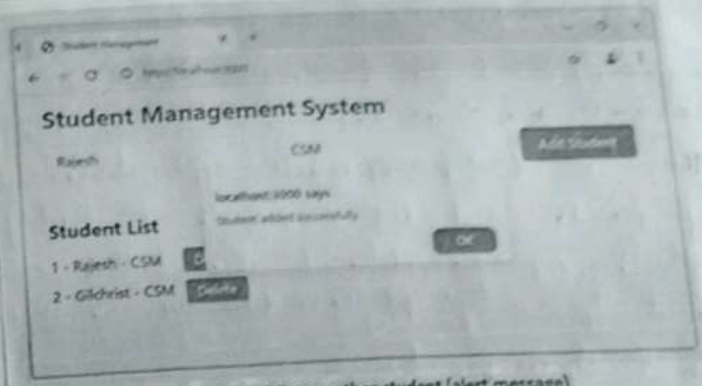
2. Initial Output in Browser (no students)



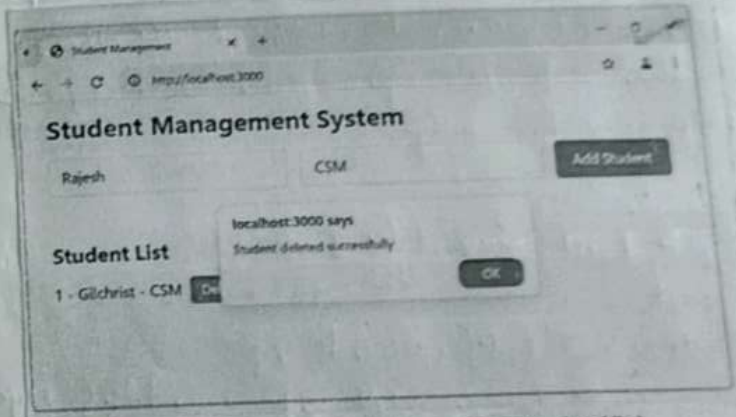
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

EXPERIMENT - 07

Create a functional component and class Component using react.

Aim :- To develop a react application using functional Components and class components to display student details, manage state, handle button clicks and dynamically update data on a webpage

• functional Component

A functional component is a javascript function that returns JSX to display the user interface. It uses React Hooks such as useState to manage state and update the webpage dynamically.

program

```
import React, {useState} from
'react';
function StudentFunctional() {
    const [name, setName] = useState("Rajesh");
    Const [course, setCourse] = useState("CSM")
    Const [count, setCount] = useState(0);
    Const changeName = () => {
            setName("Raj");
    };
    Const changeCourse = () => {
            setCourse("CSD")
    };
    Const incrementCount = () => {
            setCount(count+1);
    };
    return (
        <div style = {{
            border : "2px solid blue",
            padding : "20px",
            margin : "20px",
            border radius : "10px";
            text Align : "center"
        }}>
        <h1> Student Details </h1>
        <h2> Functional Component </h2>
        <p><b> Name : </b> {name} </p>
        <p> <b> Course : </b> {course} </p>
        <p> <b> Button clicks : </b> {count} </p>
        <button onClick = {changeName} >
            Change Name </button>
        <button onClick = {changeCourse} change
            Course </button>
        <button onClick = {incrementCount} >Increment
            </button>
        </div>
    );
```

Output:-

When the user clicks the buttons, the name, Course and Count change dynamically without reloading the webpage

Class Component:

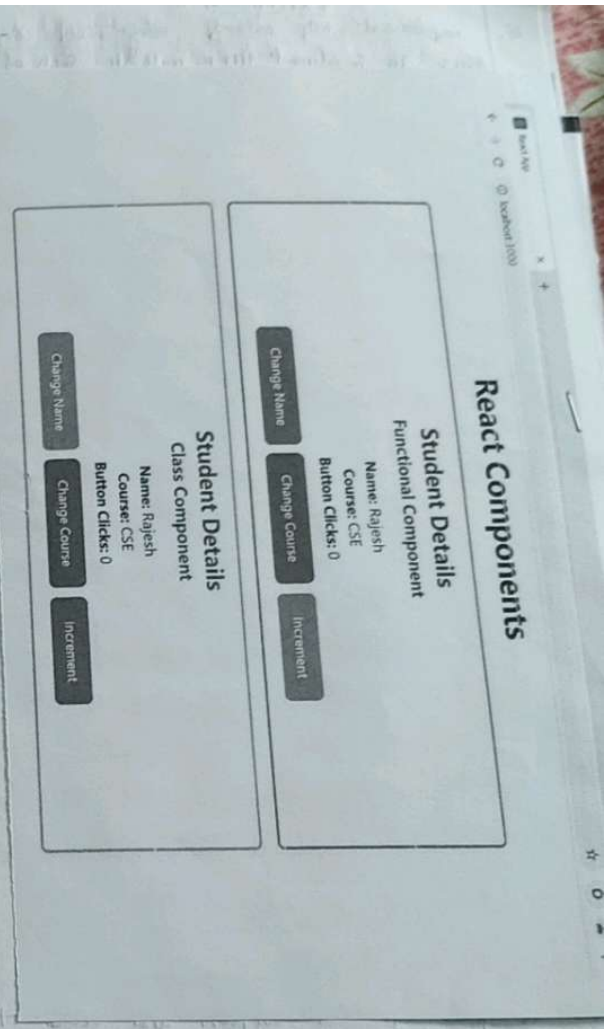
A class component is an ES6 javascript class that extends React.Component. It uses the render() method to return JSX and manages state using this.state and this.setState().

program

```
import React, {Component} from
'react';
class Student Class Extends Component {
    Constructor (props) {
        Super (props);
        this.state = {
            name : "Rajesh",
            Course : "Computer Science",
            Count : 0
        };
    }
    Change Name = () => {
        this.setState ({
            name : "Raj"
        });
    };
    Change Course = () => {
        this.setState ({
            Course : "CSM"
        });
    };
    increment Count = () => {
        this.setState ((prev State) => ({
            Count : prev State.Count + 1
        }));
    };
```

```
render () {
  return (
    <div style={{
      border: "2px solid green",
      padding: "20px",
      margin: "20px",
      borderRadius: "10px",
      textAlign: "center"
    }}>
      <h1> Student Details </h1>
      <h2> Class Component </h2>
    <p><b> Name: </b> {this.state.name}
    <p><b> Course: </b> {this.state.course}
    <p><b> Button clicks: </b> {this.state.count}
                                </p>
    <button onClick={this.changeName}>
          Change Name </button>
    <button onClick={this.changeCourse}>
          Change Course </button>
    <button onClick={this.incrementCount}>
          Increment </button>
    </div>
```

output:-



08. EXPERIMENT-08

Implement http methods using JSON server

Aim:- To implement HTTP methods such as GET, POST, PUT, PATCH and DELETE using JSON server to perform CRUD (Create, Read, Update, and delete) operations on students records.

Theory:- JSON server is a tool used to create a fake REST API using a JASON file. It allows us to perform HTTP operations without developing a backend server from Scratch. HTTP methods are used to communicate between a client and server

- GET - Retrieves data from the server
- POST - Adds new data to the server
- PUT - Replaces an existing record
- PATCH - updates selected fields of a record
- DELETE - Removes a record from the server

a. Create a JSON server

Step1:- Initialize the project

open the vsCode terminal and execute the following commands:

```
mkdir JSONserver
Cd JSONserver
npm init -y
npm install json server
```

Step2: Create db.json

Create a file named db.json and add the following Student data.

```
{ "Student": [
    {
      "id": "1",
      "name": "Rajesh",
      "Course": "CSM",
      "marks": 90
    },
    { "id": "2"
      "name": "ChrisGyle",
      "Course": "CSM",
      "marks": 70
    }
    { "id": "3"
      "name": "ABdevillers"
      "Course": "ST",
      "marks": 88
    }
```

Step 3: Start the JSON server

For the current JSON server version, execute:

npx json-server db.json

The server start at:

http://localhost:3000

The Student records can be accessed using

https://localhost:3000/students

Q. Implementation of HTTP methods

a. GET method - The GET method is used to retrieve student records from JSON server

Code:-

```
fetch ('http://localhost:3000/students')
  .then (response => response.json ())
  .then (data => {
        console.log(data);
    })
  .catch (error => {
        Console.log(error);
    });
```

Output:-

B. POST METHOD :- The POST method is used to insert a new student record into the server

Code:-

```
fetch ("http://localhost:3000/students", {
    method : 'POST',
    header : {
      'Content-Type' : application/json'
    },
    body JSON.stringify( {
      name : 'Gill',
      course : 'CSE'
      marks : 92
    })
  })
  .then (response => response.json())
  .then (data => [
        console.log (data);
  })
  . catch (error => {
        Console.log (error);
  });
```

Output:-

C. PUT method :- The PUT method is used to replace an existing student records with updated data

Code :-

```
fetch ('http://localhost:3000/students/1', {
    method: 'PUT',
    headers: {
        'Content-Type': 'application/json'
    },
    body: JSON.stringify({
        name: 'Rajesh',
        course: 'ASML',
        marks: 95
    })
})
.then (response => response.json())
.then (data => {
    console.log(data);
})
.catch (error => {
    console.log(error);
});
```

Output :-

D. PATCH method :- The PATCH method is used to update only selected fields of an existing student record

Code :-

```
fetch ('http://localhost:3000/students/2', {
    method: 'PATCH',
    headers: {
        'Content-Type': 'application/json'
    },
    body: JSON.stringify({
        marks: 96
    })
})
.then (response => response.json())
.then (data => {
    console.log(data);
})
.catch (error => {
    console.log(error);
});
```

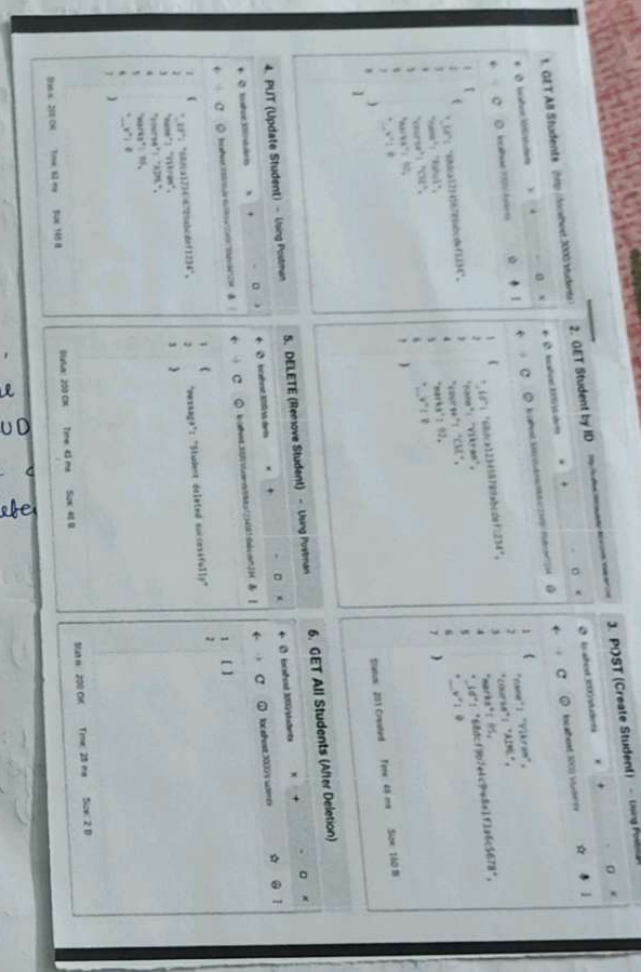
Output :-

E. DELETE METHOD :- The DELETE method is used remove an existing student record from the server.

code :-
```
fetch ('http://localhost:3000/students/3', {
    method : 'DELETE'
})
.then (response => {
    if (response.OK) {
        console.log ("student deleted Successfully")
    }
})
.catch (error => {
    console.log (error)
});
```

- After executing the POST, PUT, PATCH and DELETE operations in sequence, the remaining student will be

OUTPUT



Thus, were CRUD the delete

EXPERIMENT-09

Develop CRUD operation using mongoDB.

Aim:- To develop a web application using Node.js, Express.js and MongoDB to perform CRUD operations (create, read, update, delete) on student records

Theory:- MongoDB is NoSQL database that stores data in flexiable - JSON - like documents, CRUD ops are used to manage data in a database

Mongoose is an object Data modeling (ODM) library for MongoDB and Node.js it helps define Schemas, interact with collections and perform database operations.

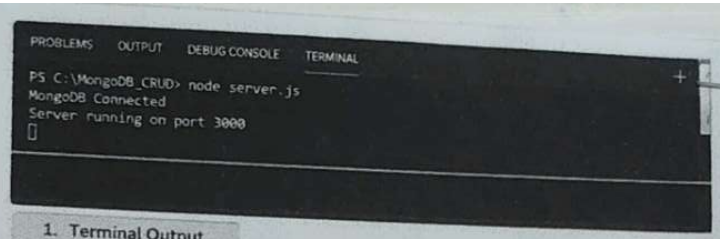
1. Installation and project setup:

```
mkdir mongo DB-CRUD
cd mongoDB-CRUD
npm init -y
npm install express mongoose
Const express = require ('express');
Const mongoose = require ('mongoose');
Const app = express ();
Const PORT = 3000;
app.use (express.json ());
Mongoose.connect ('mongoose://127.0.0.1:27017/
                                    College)
.then (() => console.log ("mongoDB connected"))
.catch (err => console.log (err));
// Create Student
Const Student Schema = new
Mongoose.Schema ({
    Name : {
      type: string
      required : true
    },
    Course : {
      type : String,
      required : true
    },
    marks : {
      type : Number
      required : true
    }
});
// Create Student model
Const Student =
    mongoose.model ('Student', Student Schema)
// CREATE : Add
app.post ('/students', async (req, res) => {
  try {
    Const student = new Student (req.body);
    Const SavedStudent = await student.save ()
  res.status (201).json (SavedStudent);
  } catch (err) {
    res.status (400).json ({message
                    : err.message });
  }
});
// READ : Get all students
app.get ('/students', async (req, res) => {
    try {
      Const students = await
```

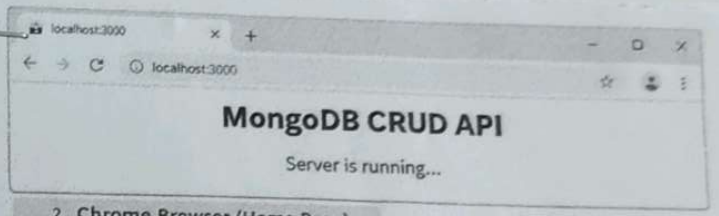
```
        Student.find();
        res.status(500).json({message: err.message})
    }
});
// READ : Get Student by ID
app.get('/Students/:id', asyn (req, res) => {
    try{
        Const Student = await
        Student.findById(req.params.id);
        if (!Student){
            rehurn res.status(404).json({
                message: "Student not found"
            });
        }
    res.status(200).json(Student)
    } Catch (err){
        res.status(400).json{{ message:
            err.message });
    }
});
//UPDATE
app.put('/Students/:id.async(req.res) => {
    try{.
    Const Student = await Student.findById
    and update(
        req.params.id
        req.body
    { new.true, runvalid a toy: true
});
if (!Student){
    return res: status(404).json({
        message: "Students and found"
    });
});
}
res.status(200).json(Student);
} Catch (err){
    res.status(400).json({message: err.message});
}
```

Delete

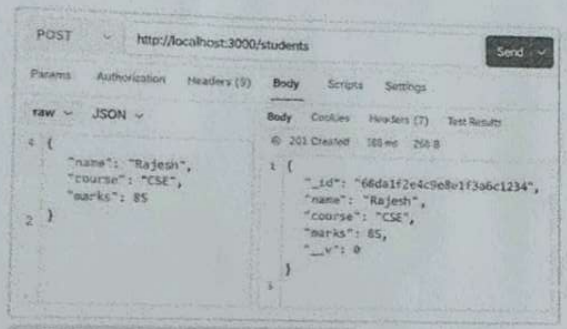
```
app.delete('/student/:id: async(req, res) => {
    try{
    Const Student = await Student.findById and
    Delete(req.params.id);
    if (!Student){
        rehurn res.status(404).json({
        message: "Student not found"
    });
}
res.status(200).json({
        message: "Student elelted successfully
    });
}
Catch (err){
    res.status(400).json({message.
        err.message});
}
});
```



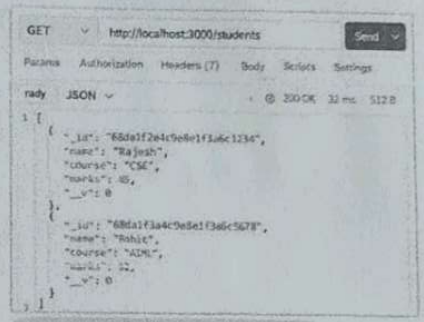
1. Terminal Output



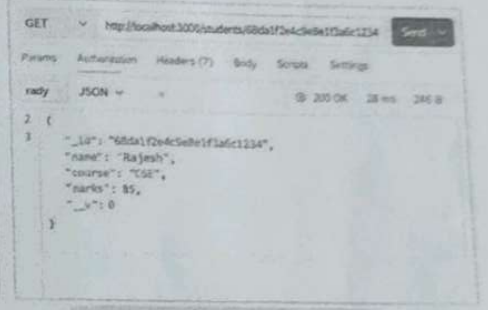
2. Chrome Browser (Home Page)



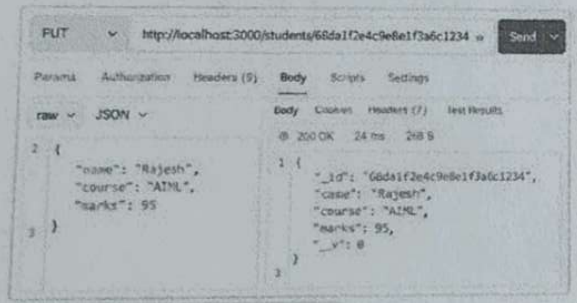
3. Create (POST) – Add Student



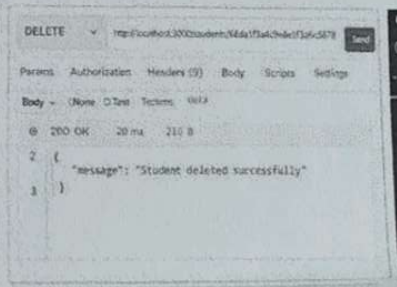
4. Read (GET) – Get All Students



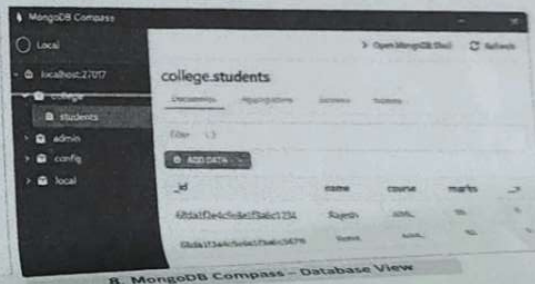
5. Read (GET) – Get Student by ID



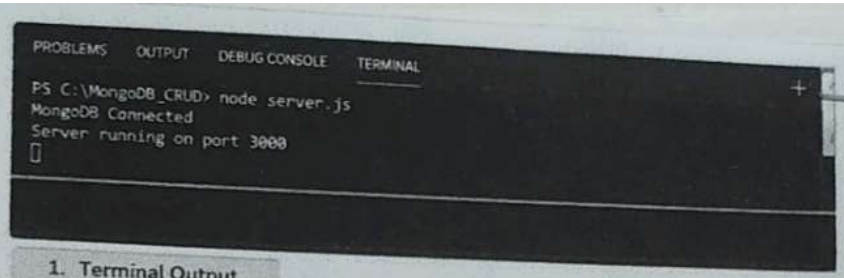
6. Update (PUT) – Update Student



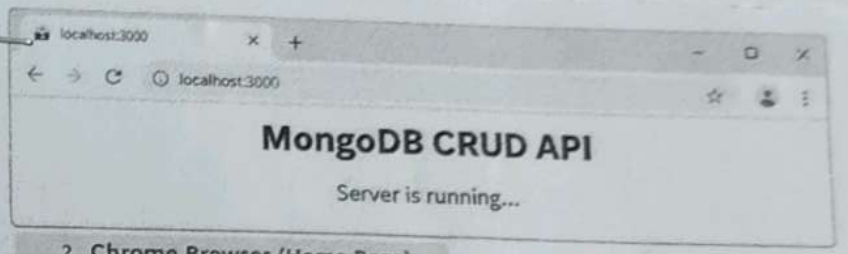
7. Delete (DELETE) – Remove Student



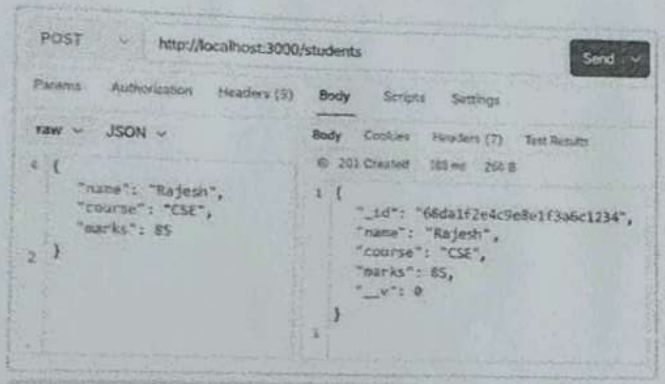
8. MongoDB Compass – Database View



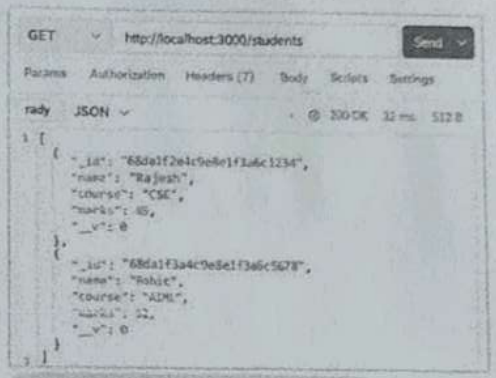
1. Terminal Output



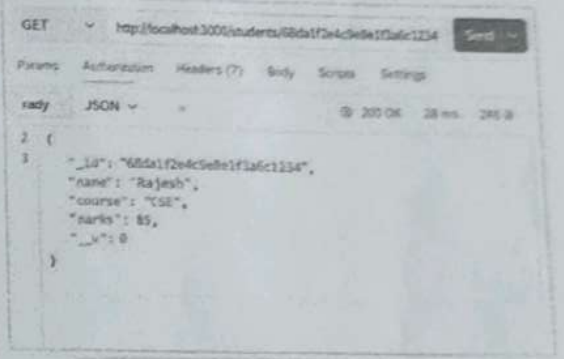
2. Chrome Browser (Home Page)



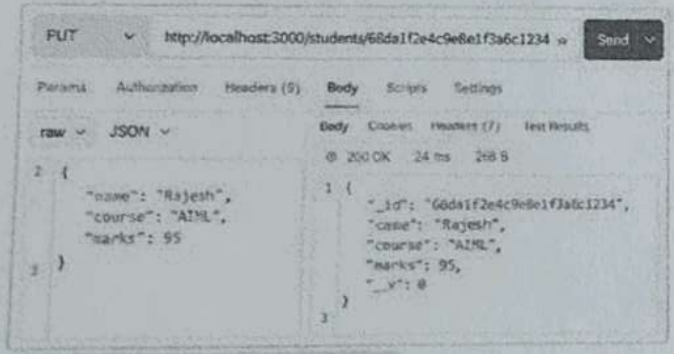
3. Create (POST) – Add Student



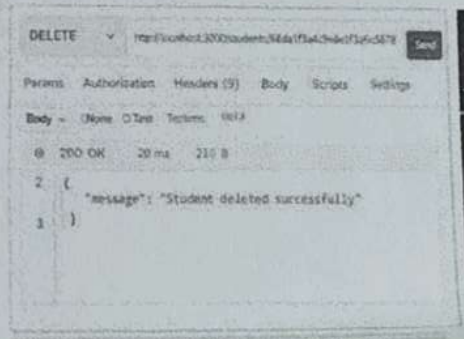
4. Read (GET) – Get All Students



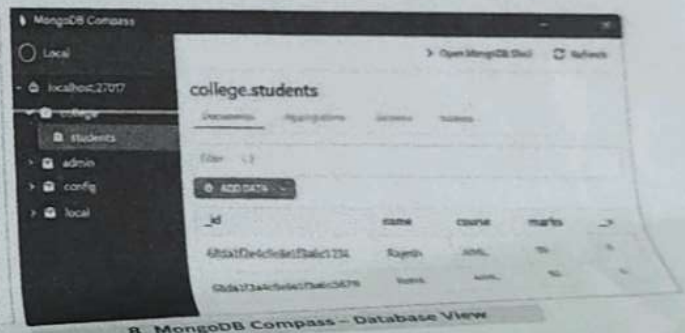
5. Read (GET) – Get Student by ID



6. Update (PUT) – Update Student



7. Delete (DELETE) – Remove Student



8. MongoDB Compass – Database View